

34.5-8

In the *half 3-CNF satisfiability* problem, the input is a 3-CNF formula ϕ with n variables and m clauses, where m is even. The question is whether there exists a truth assignment to the variables of ϕ such that exactly half the clauses evaluate to 0 and exactly half the clauses evaluate to 1. Prove that the half 3-CNF satisfiability problem is NP-complete.

34.5-9

The proof that VERTEX-COVER $\leq_P$ HAM-CYCLE assumes that the graph G given as input to the vertex-cover problem has no isolated vertices. Show how the reduction in the proof can break down if G has an isolated vertex.

Problems
34-1 Independent set

An *independent set* of a graph $G = (V, E)$ is a subset $V' \subseteq V$ of vertices such that each edge in E is incident on at most one vertex in V' . The *independent-set problem* is to find a maximum-size independent set in G .

- a. Formulate a related decision problem for the independent-set problem, and prove that it is NP-complete. (*Hint:* Reduce from the clique problem.)
- b. You are given a “black-box” subroutine to solve the decision problem you defined in part (a). Give an algorithm to find an independent set of maximum size. The running time of your algorithm should be polynomial in $|V|$ and $|E|$, counting queries to the black box as a single step.

Although the independent-set decision problem is NP-complete, certain special cases are polynomial-time solvable.

- c. Give an efficient algorithm to solve the independent-set problem when each vertex in G has degree 2. Analyze the running time, and prove that your algorithm works correctly.
- d. Give an efficient algorithm to solve the independent-set problem when G is bipartite. Analyze the running time, and prove that your algorithm works correctly. (*Hint:* First prove that in a bipartite graph, the size of the maximum independent set plus the size of the maximum matching is equal to $|V|$. Then use a maximum-matching algorithm (see Section 25.1) as a first step in an algorithm to find an independent set.)

34-2 Bonnie and Clyde

Bonnie and Clyde have just robbed a bank. They have a bag of money and want to divide it up. For each of the following scenarios, either give a polynomial-time algorithm to divide the money or prove that the problem of dividing the money in the manner described is NP-complete. The input in each case is a list of the n items in the bag, along with the value of each.

- a. The bag contains n coins, but only two different denominations: some coins are worth x dollars, and some are worth y dollars. Bonnie and Clyde wish to divide the money exactly evenly.
- b. The bag contains n coins, with an arbitrary number of different denominations, but each denomination is a nonnegative exact power of 2, so that the possible denominations are 1 dollar, 2 dollars, 4 dollars, etc. Bonnie and Clyde wish to divide the money exactly evenly.
- c. The bag contains n checks, which are, in an amazing coincidence, made out to “Bonnie or Clyde.” They wish to divide the checks so that they each get the exact same amount of money.
- d. The bag contains n checks as in part (c), but this time Bonnie and Clyde are willing to accept a split in which the difference is no larger than 100 dollars.

34-3 Graph coloring

Mapmakers try to use as few colors as possible when coloring countries on a map, subject to the restriction that if two countries share a border, they must have different colors. You can model this problem with an undirected graph $G = (V, E)$ in which each vertex represents a country and vertices whose respective countries share a border are adjacent. Then, a ***k-coloring*** is a function $c : V \rightarrow \{1, 2, \dots, k\}$ such that $c(u) \neq c(v)$ for every edge $(u, v) \in E$. In other words, the numbers $1, 2, \dots, k$ represent the k colors, and adjacent vertices must have different colors. The ***graph-coloring problem*** is to determine the minimum number of colors needed to color a given graph.

- a. Give an efficient algorithm to determine a 2-coloring of a graph, if one exists.
- b. Cast the graph-coloring problem as a decision problem. Show that your decision problem is solvable in polynomial time if and only if the graph-coloring problem is solvable in polynomial time.
- c. Let the language 3-COLOR be the set of graphs that can be 3-colored. Show that if 3-COLOR is NP-complete, then your decision problem from part (b) is NP-complete.

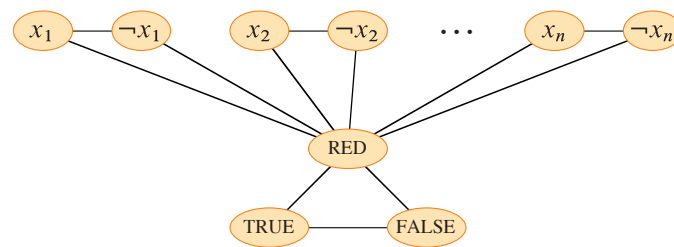


Figure 34.20 The subgraph of G in Problem 34-3 formed by the literal edges. The special vertices TRUE, FALSE, and RED form a triangle, and for each variable x_i , the vertices x_i , $\neg x_i$, and RED form a triangle.

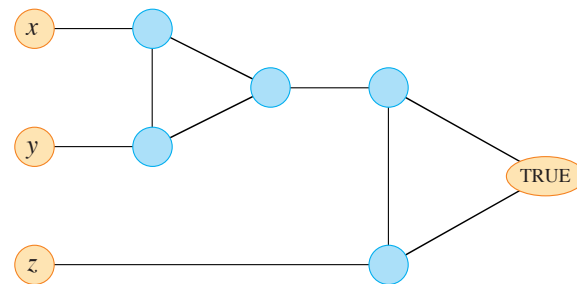


Figure 34.21 The gadget corresponding to a clause $(x \vee y \vee z)$, used in Problem 34-3.

To prove that 3-COLOR is NP-complete, you can reduce from 3-CNF-SAT. Given a formula ϕ of m clauses on n variables $x_1, x_2, \dots, x_n$, construct a graph $G = (V, E)$ as follows. The set V consists of a vertex for each variable, a vertex for the negation of each variable, five vertices for each clause, and three special vertices: TRUE, FALSE, and RED. The edges of the graph are of two types: “literal” edges that are independent of the clauses and “clause” edges that depend on the clauses. As Figure 34.20 shows, the literal edges form a triangle on the three special vertices TRUE, FALSE, and RED, and they also form a triangle on x_i , $\neg x_i$, and RED for $i = 1, 2, \dots, n$.

- d.* Consider a graph containing the literal edges. Argue that in any 3-coloring c of such a graph, exactly one of a variable and its negation is colored $c(\text{TRUE})$ and the other is colored $c(\text{FALSE})$. Then argue that for any truth assignment for ϕ , there exists a 3-coloring of the graph containing just the literal edges.

The gadget shown in Figure 34.21 helps to enforce the condition corresponding to a clause $(x \vee y \vee z)$, where x , y , and z are literals. Each clause requires a unique copy of the five blue vertices in the figure. They connect as shown to the literals of the clause and the special vertex TRUE.

- e. Argue that if each of x , y , and z is colored $c(\text{TRUE})$ or $c(\text{FALSE})$, then the gadget is 3-colorable if and only if at least one of x , y , or z is colored $c(\text{TRUE})$.
- f. Complete the proof that 3-COLOR is NP-complete.

34-4 Scheduling with profits and deadlines

You have one computer and a set of n tasks $\{a_1, a_2, \dots, a_n\}$ requiring time on the computer. Each task a_j requires t_j time units on the computer (its processing time), yields a profit of p_j , and has a deadline d_j . The computer can process only one task at a time, and task a_j must run without interruption for t_j consecutive time units. If task a_j completes by its deadline d_j , you receive a profit p_j . If instead task a_j completes after its deadline, you receive no profit. As an optimization problem, given the processing times, profits, and deadlines for a set of n tasks, you wish to find a schedule that completes all the tasks and returns the greatest amount of profit. The processing times, profits, and deadlines are all nonnegative numbers.

- a. State this problem as a decision problem.
- b. Show that the decision problem is NP-complete.
- c. Give a polynomial-time algorithm for the decision problem, assuming that all processing times are integers from 1 to n . (*Hint: Use dynamic programming.*)
- d. Give a polynomial-time algorithm for the optimization problem, assuming that all processing times are integers from 1 to n .

Chapter notes

The book by Garey and Johnson [176] provides a wonderful guide to NP-completeness, discussing the theory at length and providing a catalogue of many problems that were known to be NP-complete in 1979. The proof of Theorem 34.13 is adapted from their book, and the list of NP-complete problem domains at the beginning of Section 34.5 is drawn from their table of contents. Johnson wrote a series of 23 columns in the *Journal of Algorithms* between 1981 and 1992 reporting new developments in NP-completeness. Fortnow's book [152] gives a history of NP-completeness, along with societal implications. Hopcroft, Motwani, and Ullman [225], Lewis and Papadimitriou [299], Papadimitriou [352], and Sipser [413] have good treatments of NP-completeness in the context of complexity theory. NP-completeness and several reductions also appear in books by Aho, Hopcroft, and Ullman [5], Dasgupta, Papadimitriou, and Vazirani [107], Johnsonbaugh and

Schaefer [239], and Kleinberg and Tardos [257]. The book by Hromkovič [229] studies various methods for solving hard problems.

The class P was introduced in 1964 by Cobham [96] and, independently, in 1965 by Edmonds [130], who also introduced the class NP and conjectured that $P \neq NP$. The notion of NP-completeness was proposed in 1971 by Cook [100], who gave the first NP-completeness proofs for formula satisfiability and 3-CNF satisfiability. Levin [297] independently discovered the notion, giving an NP-completeness proof for a tiling problem. Karp [248] introduced the methodology of reductions in 1972 and demonstrated the rich variety of NP-complete problems. Karp's paper included the original NP-completeness proofs of the clique, vertex-cover, and hamiltonian-cycle problems. Since then, thousands of problems have been proven to be NP-complete by many researchers.

Work in complexity theory has shed light on the complexity of computing approximate solutions. This work gives a new definition of NP using “probabilistically checkable proofs.” This new definition implies that for problems such as clique, vertex cover, the traveling-salesperson problem with the triangle inequality, and many others, computing good approximate solutions (see Chapter 35) is NP-hard and hence no easier than computing optimal solutions. An introduction to this area can be found in Arora's thesis [21], a chapter by Arora and Lund in Hochbaum [221], a survey article by Arora [22], a book edited by Mayr, Prömel, and Steger [319], a survey article by Johnson [237], and a chapter in the textbook by Arora and Barak [24].

Many problems of practical significance are NP-complete, yet they are too important to abandon merely because nobody knows how to find an optimal solution in polynomial time. Even if a problem is NP-complete, there may be hope. You have at least three options to get around NP-completeness. First, if the actual inputs are small, an algorithm with exponential running time might be fast enough. Second, you might be able to isolate important special cases that you can solve in polynomial time. Third, you can try to devise an approach to find a *near-optimal* solution in polynomial time (either in the worst case or the expected case). In practice, near-optimality is often good enough. We call an algorithm that returns near-optimal solutions an *approximation algorithm*. This chapter presents polynomial-time approximation algorithms for several NP-complete problems.

Performance ratios for approximation algorithms

Suppose that you are working on an optimization problem in which each potential solution has a positive cost, and you want to find a near-optimal solution. Depending on the problem, you could define an optimal solution as one with maximum possible cost or as one with minimum possible cost, which is to say that the problem might be either a maximization or a minimization problem.

We say that an algorithm for a problem has an *approximation ratio* of $\rho(n)$ if, for any input of size n , the cost C of the solution produced by the algorithm is within a factor of $\rho(n)$ of the cost C^* of an optimal solution:

$$\max \left\{ \frac{C}{C^*}, \frac{C^*}{C} \right\} \leq \rho(n) \quad (35.1)$$

If an algorithm achieves an approximation ratio of $\rho(n)$, we call it a *$\rho(n)$ -approximation algorithm*. The definitions of approximation ratio and $\rho(n)$ -approximation algorithm apply to both minimization and maximization problems. For a maximization problem, $0 < C \leq C^*$, and the ratio C^*/C gives the factor by which

the cost of an optimal solution is larger than the cost of the approximate solution. Similarly, for a minimization problem, $0 < C^* \leq C$, and the ratio C/C^* gives the factor by which the cost of the approximate solution is larger than the cost of an optimal solution. Because we assume that all solutions have positive cost, these ratios are always well defined. The approximation ratio of an approximation algorithm is never less than 1, since $C/C^* \leq 1$ implies $C^*/C \geq 1$. Therefore, a 1-approximation algorithm¹ produces an optimal solution, and an approximation algorithm with a large approximation ratio may return a solution that is much worse than optimal.

For many problems, we know of polynomial-time approximation algorithms with small constant approximation ratios, although for other problems, the best known polynomial-time approximation algorithms have approximation ratios that grow as functions of the input size n . An example of such a problem is the set-cover problem presented in Section 35.3.

Some polynomial-time approximation algorithms can achieve increasingly better approximation ratios by using more and more computation time. For such problems, you can trade computation time for the quality of the approximation. An example is the subset-sum problem studied in Section 35.5. This situation is important enough to deserve a name of its own.

An **approximation scheme** for an optimization problem is an approximation algorithm that takes as input not only an instance of the problem, but also a value $\epsilon > 0$ such that for any fixed ϵ , the scheme is a $(1 + \epsilon)$ -approximation algorithm. We say that an approximation scheme is a **polynomial-time approximation scheme** if for any fixed $\epsilon > 0$, the scheme runs in time polynomial in the size n of its input instance.

The running time of a polynomial-time approximation scheme can increase very rapidly as ϵ decreases. For example, the running time of a polynomial-time approximation scheme might be $O(n^{2/\epsilon})$. Ideally, if ϵ decreases by a constant factor, the running time to achieve the desired approximation should not increase by more than a constant factor (though not necessarily the same constant factor by which ϵ decreased).

We say that an approximation scheme is a **fully polynomial-time approximation scheme** if it is an approximation scheme and its running time is polynomial in both $1/\epsilon$ and the size n of the input instance. For example, the scheme might have a running time of $O((1/\epsilon)^2 n^3)$. With such a scheme, any constant-factor decrease in ϵ comes with a corresponding constant-factor increase in the running time.

¹ When the approximation ratio is independent of n , we use the terms “approximation ratio of ρ ” and “ ρ -approximation algorithm,” indicating no dependence on n .

Chapter outline

The first four sections of this chapter present some examples of polynomial-time approximation algorithms for NP-complete problems, and the fifth section gives a fully polynomial-time approximation scheme. We begin in Section 35.1 with a study of the vertex-cover problem, an NP-complete minimization problem that has an approximation algorithm with an approximation ratio of 2. Section 35.2 looks at a version of the traveling-salesperson problem in which the cost function satisfies the triangle inequality and presents an approximation algorithm with an approximation ratio of 2. The section also shows that without the triangle inequality, for any constant $\rho \geq 1$, a ρ -approximation algorithm cannot exist unless $P = NP$. Section 35.3 applies a greedy method as an effective approximation algorithm for the set-covering problem, obtaining a covering whose cost is at worst a logarithmic factor larger than the optimal cost. Section 35.4 uses randomization and linear programming to develop two more approximation algorithms. The section first defines the optimization version of 3-CNF satisfiability and gives a simple randomized algorithm that produces a solution with an expected approximation ratio of $8/7$. Then Section 35.4 examines a weighted variant of the vertex-cover problem and exhibits how to use linear programming to develop a 2-approximation algorithm. Finally, Section 35.5 presents a fully polynomial-time approximation scheme for the subset-sum problem.

35.1 The vertex-cover problem

Section 34.5.2 defined the vertex-cover problem and proved it NP-complete. Recall that a **vertex cover** of an undirected graph $G = (V, E)$ is a subset $V' \subseteq V$ such that if (u, v) is an edge of G , then either $u \in V'$ or $v \in V'$ (or both). The size of a vertex cover is the number of vertices in it.

The **vertex-cover problem** is to find a vertex cover of minimum size in a given undirected graph. We call such a vertex cover an **optimal vertex cover**. This problem is the optimization version of an NP-complete decision problem.

Even though nobody knows how to find an optimal vertex cover in a graph G in polynomial time, there is an efficient algorithm to find a vertex cover that is near-optimal. The approximation algorithm APPROX-VERTEX-COVER on the facing page takes as input an undirected graph G and returns a vertex cover whose size is guaranteed to be no more than twice the size of an optimal vertex cover.

Figure 35.1 illustrates how APPROX-VERTEX-COVER operates on an example graph. The variable C contains the vertex cover being constructed. Line 1 initializes C to the empty set. Line 2 sets E' to be a copy of the edge set $G.E$ of the graph. The **while** loop of lines 3–6 repeatedly picks an edge (u, v) from E' , adds


```

APPROX-VERTEX-COVER( $G$ )
1   $C = \emptyset$ 
2   $E' = G.E$ 
3  while  $E' \neq \emptyset$ 
4      let  $(u, v)$  be an arbitrary edge of  $E'$ 
5       $C = C \cup \{u, v\}$ 
6      remove from  $E'$  edge  $(u, v)$  and every edge incident on either  $u$  or  $v$ 
7  return  $C$ 

```

its endpoints u and v into C , and deletes all edges in E' that u or v covers. Finally, line 7 returns the vertex cover C . The running time of this algorithm is $O(V + E)$, using adjacency lists to represent E' .

Theorem 35.1

APPROX-VERTEX-COVER is a polynomial-time 2-approximation algorithm.

Proof We have already shown that APPROX-VERTEX-COVER runs in polynomial time.

The set C of vertices that is returned by APPROX-VERTEX-COVER is a vertex cover, since the algorithm loops until every edge in $G.E$ has been covered by some vertex in C .

To see that APPROX-VERTEX-COVER returns a vertex cover that is at most twice the size of an optimal cover, let A denote the set of edges that line 4 of APPROX-VERTEX-COVER picked. In order to cover the edges in A , any vertex cover—in particular, an optimal cover C^* —must include at least one endpoint of each edge in A . No two edges in A share an endpoint, since once an edge is picked in line 4, all other edges that are incident on its endpoints are deleted from E' in line 6. Thus, no two edges in A are covered by the same vertex from C^* , meaning that for every vertex in C^* , there is at most one edge in A , giving the lower bound

$$|C^*| \geq |A| \tag{35.2}$$

on the size of an optimal vertex cover. Each execution of line 4 picks an edge for which neither of its endpoints is already in C , yielding an upper bound (an exact upper bound, in fact) on the size of the vertex cover returned:

$$|C| = 2|A|. \tag{35.3}$$

Combining equations (35.2) and (35.3) yields

$$\begin{aligned}
 |C| &= 2|A| \\
 &\leq 2|C^*|,
 \end{aligned}$$

thereby proving the theorem. ■

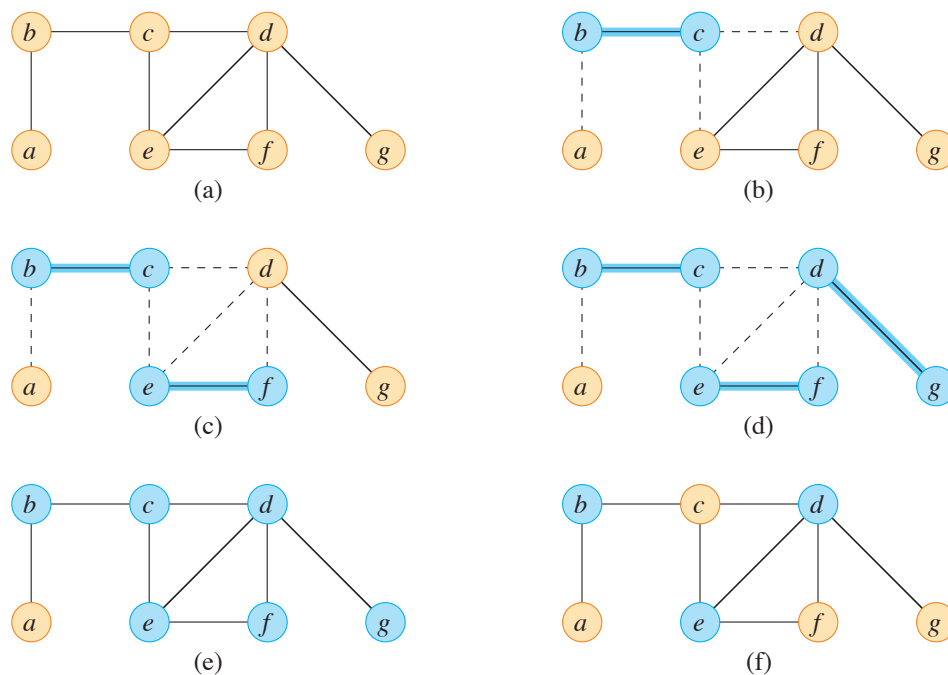


Figure 35.1 The operation of APPROX-VERTEX-COVER. (a) The input graph G , which has 7 vertices and 8 edges. (b) The highlighted edge (b, c) is the first edge chosen by APPROX-VERTEX-COVER. Vertices b and c , in blue, are added to the set C containing the vertex cover being created. Dashed edges (a, b) , (c, e) , and (c, d) are removed since they are now covered by some vertex in C . (c) Edge (e, f) is chosen, and vertices e and f are added to C . (d) Edge (d, g) is chosen, and vertices d and g are added to C . (e) The set C , which is the vertex cover produced by APPROX-VERTEX-COVER, contains the six vertices b, c, d, e, f, g . (f) The optimal vertex cover for this problem contains only three vertices: b, d , and e .

Let us reflect on this proof. At first, you might wonder how you can possibly prove that the size of the vertex cover returned by APPROX-VERTEX-COVER is at most twice the size of an optimal vertex cover, when you don't even know the size of an optimal vertex cover. Instead of requiring that you know the exact size of an optimal vertex cover, you find a lower bound on the size. As Exercise 35.1-2 asks you to show, the set A of edges that line 4 of APPROX-VERTEX-COVER selects is actually a maximal matching in the graph G . (A *maximal matching* is a matching to which no edges can be added and still have a matching.) The size of a maximal matching is, as we argued in the proof of Theorem 35.1, a lower bound on the size of an optimal vertex cover. The algorithm returns a vertex cover whose size is at most twice the size of the maximal matching A . The approximation ratio comes from relating the size of the solution returned to the lower bound. We will use this methodology in later sections as well.